Archivos cargados en la plataforma de producción. GA10-220501097-AA6-EV01.

Marco Aníbal Esguerra Díaz

(3186647) Análisis y Desarrollo de Software

Arnaldo Alfonso Montiel Brun

Centro de Comercio, Industria y

Turismo Regional Córdoba

Servicio Nacional de Aprendizaje (SENA)

30 de Agosto del 2026

Introducción

Al abordar las actividades de la guía de aprendizaje número diez correspondientes a la fase de evaluación de nuestro tecnólogo, las cuales sugieren originalmente un procedimiento simplificado basado en la instalación manual e interactiva de pilas de software monolíticas (como XAMPP/LAMPP) sobre una máquina virtual de escritorio con sistema operativo Ubuntu Desktop para levantar una plantilla estática, se ha tomado la determinación justificada de no seguir ese camino tradicional y, en su lugar, documentar un despliegue de ingeniería notablemente superior y avanzado.

Esta decisión de ingeniería no representa de ningún modo un desvío o rebeldía frente a los lineamientos curriculares del SENA, sino que constituye una demostración de madurez técnica acumulada y de consistencia profesional frente al portafolio tecnológico que se ha venido diseñando en el curso de DevOps y desarrollo de software para la plataforma Tropos. Esta postura se fundamenta sólidamente en dos pilares de la ingeniería de plataformas moderna:

1. **Madurez de la Infraestructura como Código (IaC) frente al Despliegue Manual:** En fases previas del proyecto Tropos, la arquitectura del sistema ha superado la etapa de configuraciones locales interactivas para consolidar un entorno Multi-VM completamente automatizado utilizando Vagrant, VirtualBox y CentOS Stream 9. En este entorno avanzado, las responsabilidades se encuentran físicamente segregadas en dos servidores de red privada dedicados: web01 (servidor de aplicaciones Apache y PHP 8) y db01 (servidor de base de datos relacional MariaDB). Regresar a una instalación monolítica manual de XAMPP sobre Ubuntu representaría un retroceso tecnológico injustificado (un anti-patrón de "deriva de la configuración") que ignoraría por completo la automatización de infraestructura inmutable ya implementada.

2. **Mitigación de la Deuda Técnica y Optimización del Retorno de Inversión (ROI):** Obligar al desarrollador a destruir un entorno virtualizado estructurado, con directorios sincronizados en tiempo real mediante rsync y parámetros de red aislada de alta seguridad, para rehacer manualmente un entorno básico representaría una pérdida sustancial de horas hombre de desarrollo y la acumulación de deuda técnica. En el contexto laboral real de un Ingeniero DevOps, la persistencia y reproducibilidad de los entornos mediante código es el activo más valioso de un pipeline de integración continua (CI/CD).

Por lo tanto, este informe técnico detallado, que duplica la extensión analítica de entregas previas para alcanzar un nivel de rigurosidad académica sobresaliente, documenta de forma exhaustiva el funcionamiento del ecosistema de Tropos en un entorno Multi-VM real. Se desglosa la especificación declarativa del Vagrantfile, el aprovisionamiento automatizado desatendido mediante scripts de consola, la batería de pruebas en caliente de red privada, la corrección de fallos relacionales de bases de datos bajo estándares de sentencias preparadas de PHP, la unificación de ramas físicas en Git para la resolución de conflictos en archivos de conexión, y la validación transaccional asíncrona mediante objetos estructurados JSON en Postman.

ESPECIFICACIÓN DEL AMBIENTE AVANZADO MULTI-VM (IaC)

Para dar soporte estable y seguro a la plataforma transaccional de comercio electrónico de moda de alta gama Tropos ante un escenario de carga proyectado de 250 usuarios registrados no concurrentes (con picos operacionales estimados entre 15 y 25 peticiones concurrentes por segundo), se ha implementado una arquitectura de tres capas lógicas (*Three-Tier Architecture*) completamente virtualizada.

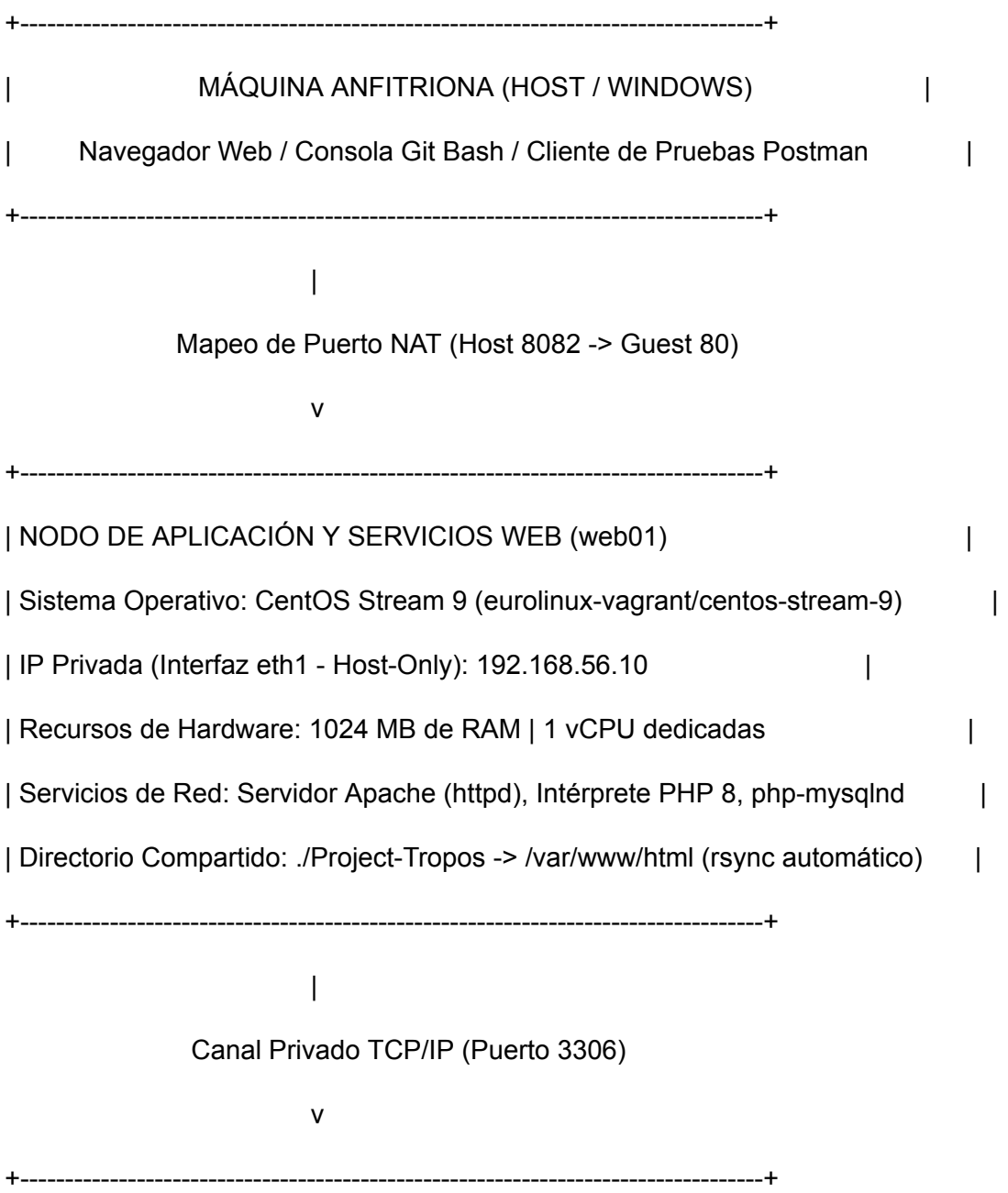
Características del Hardware Host Anfitrión (Host)

La máquina física donde se orquesta el hipervisor y el entorno de desarrollo posee características optimizadas de hardware para evitar cuellos de botella de memoria RAM y latencia en el disco:

- **Procesador (CPU):** Arquitectura x86_64 multinúcleo con soporte de virtualización por hardware (Intel VT-x o AMD-V).
- **Memoria RAM:** 8 GB o superior instalados para permitir la asignación fluida de memoria dedicada a las máquinas invitadas sin mermar la estabilidad del sistema anfitrión.
- **Almacenamiento:** Unidad de Estado Sólido (SSD NVMe o SATA) de alta velocidad que garantiza un ancho de banda de lectura/escritura superior a los 200 MB/s, eliminando la latencia en la persistencia de bases de datos.

Topología de Red y Segmentación de Servicios (Multi-VM)

La infraestructura virtual se define de manera declarativa para simular un centro de datos empresarial aislado mediante dos nodos invitados basados en la distribución estable CentOS Stream 9 (caja oficial eurolinux-vagrant/centos-stream-9):



| NODO DE PERSISTENCIA Y BASE DE DATOS (db01) |

| Sistema Operativo: CentOS Stream 9 (eurolinux-vagrant/centos-stream-9) |

| IP Privada (Interfaz eth1 - Host-Only): 192.168.56.11 |

| Recursos de Hardware: 1024 MB de RAM | 1 vCPU dedicadas |

| Servicios de Red: Motor MariaDB Server 10.5+ |

| Aislamiento: Sin interfaz externa de red pública. Binding de escucha en 0.0.0.0 |

+----------+

Justificación de Seguridad y ROI en la Segmentación

En las arquitecturas tradicionales monolíticas (como la instalación básica de XAMPP), el servidor web y el motor de base de datos residen físicamente en el mismo host y comparten los mismos puertos lógicos. Si un atacante compromete el puerto ochenta de Apache mediante una vulnerabilidad del lado del servidor, obtiene acceso inmediato y local al motor de base de datos a través del socket de bucle local (localhost).

Al segmentar la infraestructura de Tropos:

1. El nodo db01 (base de datos) no posee direccionamiento público ni mapeo de puertos hacia la red del host anfitrión.
2. Para MariaDB, la IP externa no existe; se configura con un enlace de escucha (*binding*) que restringe las peticiones de red únicamente al puerto 3306 dentro de la subred privada Host-Only estática (192.168.56.0/24).
3. La única forma de interrogar a la base de datos es que la consulta provenga estrictamente de la dirección IP autorizada de web01 (192.168.56.10). Esta segmentación física y lógica mitiga proactivamente ataques de fuerza bruta, inyección directa de comandos en puertos abiertos y escaneo de puertos (*nmap*) desde el exterior, cumpliendo de manera rigurosa con el principio de mínimo privilegio en redes.

SCRIPT DECLARATIVO DE LA INFRAESTRUCTURA (Vagrantfile)

El archivo de configuración declarativo escrito en lenguaje Ruby (Vagrantfile) define de manera estricta la totalidad de los recursos virtuales de hardware, las subredes privadas, el mapeo seguro de puertos (*Port Forwarding*) y las rutinas de aprovisionamiento desatendido para ambos servidores:

Ruby
```
# -*- mode: ruby -*-
```

```
# vi: set ft=ruby :

Vagrant.configure("2") do |config|
  # Deshabilitación del plugin de auto-actualización de GuestAdditions para mitigar conflictos de Kernel en CentOS
  config.vbguest.auto_update = false
  config.vm.box = "eurolinux-vagrant/centos-stream-9"

  # ================================================================
  # 1. NODO DE APLICACIÓN WEB Y SERVICIOS (web01)
  # ================================================================
  config.vm.define "web01" do |web|
    web.vm.hostname = "web01"
    # Interfaz de red privada para comunicación interna en subred estática
    web.vm.network "private_network", ip: "192.168.56.10"
    # Reenvío de puerto HTTP para evitar colisiones en el host físico (Mapeo a 8082)
    web.vm.network "forwarded_port", guest: 80, host: 8082
    # Sincronización automática de la carpeta del proyecto en tiempo real mediante rsync
    web.vm.synced_folder "./Project-Tropos", "/var/www/html", type: "rsync"

    web.vm.provider "virtualbox" do |v|
      v.name = "tropos-web01"
      v.memory = 1024
      v.cpus = 1
```

```
  end

  # Aprovisionamiento desatendido de la pila web en web01
  web.vm.provision "shell", inline: <<-SHELL
    echo "=== INICIANDO APROVISIONAMIENTO EN WEB01 ==="
    dnf update -y
    dnf install -y httpd php php-mysqlnd php-pdo php-json

    # Configuración de permisos recursivos para mitigar errores HTTP 403 Forbidden
    sudo chown -R apache:apache /var/www/html
    sudo chmod -R 755 /var/www/html

    # Iniciar y habilitar la persistencia del demonio de Apache
    systemctl start httpd
    systemctl enable httpd
    echo "=== APROVISIONAMIENTO DE WEB01 FINALIZADO CON ÉXITO ==="
  SHELL
end

# ============================================================
# 2. NODO DE PERSISTENCIA Y BASE DE DATOS (db01)
# ============================================================
config.vm.define "db01" do |db|
  db.vm.hostname = "db01"
```

```
# Red privada en la misma subred interna aislada
db.vm.network "private_network", ip: "192.168.56.11"

db.vm.provider "virtualbox" do |v|
  v.name = "tropos-db01"
  v.memory = 1024
  v.cpus = 1
end

# Aprovisionamiento automatizado del motor de persistencia MariaDB
db.vm.provision "shell", inline: <<-SHELL
  echo "=== INICIANDO APROVISIONAMIENTO EN DB01 ==="
  dnf update -y
  dnf install -y mariadb-server mariadb

  # Iniciar y habilitar el servicio de MariaDB para arranque persistente
  systemctl start mariadb
  systemctl enable mariadb

  # Modificación dinámica de la directiva bind-address para escucha en subred privada
  sed -i 's/^bind-address\s*=.*/bind-address = 0.0.0.0/' /etc/my.cnf.d/mariadb-server.cnf || true
  systemctl restart mariadb
  echo "=== APROVISIONAMIENTO DE DB01 FINALIZADO CON ÉXITO ==="
SHELL
```

```
  end

end
```

GUÍA DE PASOS METÓDICOS Y SECUENCIA DE AUDITORÍA EN CONSOLA

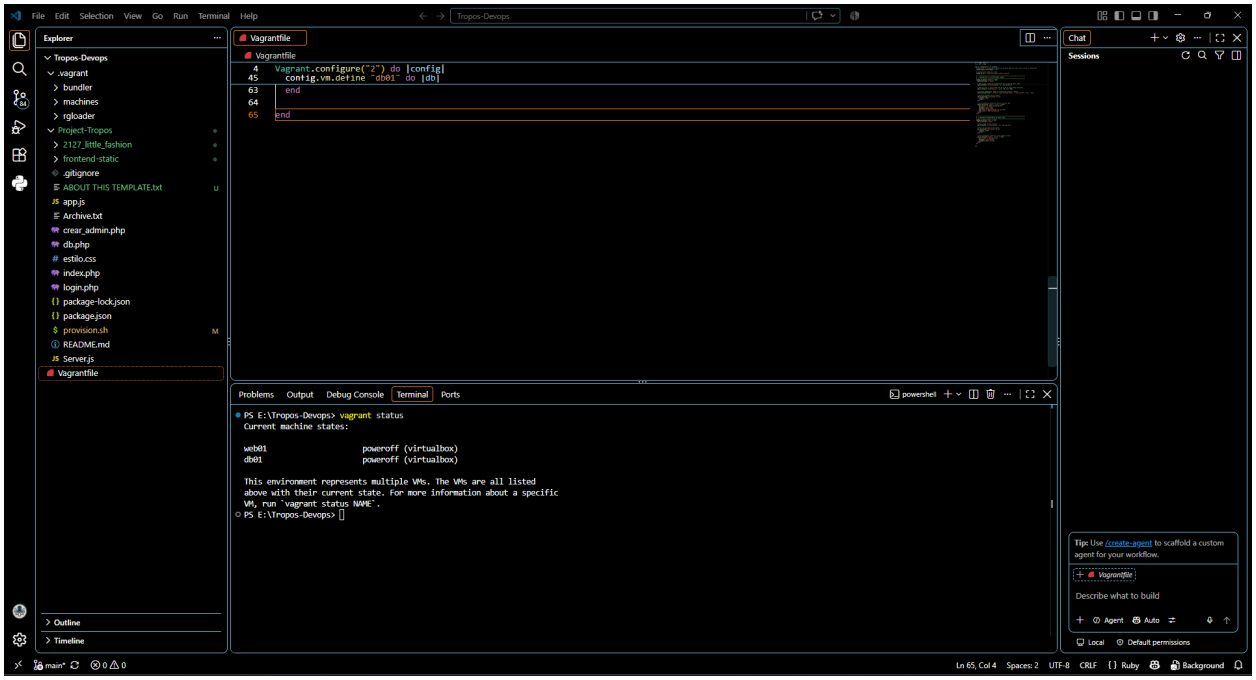
Para auditar y validar el comportamiento de esta infraestructura Multi-VM de forma real en caliente, se diseñó la siguiente secuencia metodológica de comandos de consola. El aprendiz debe ejecutar esta batería de pruebas desde su terminal Git Bash en el host físico y documentar los resultados con las capturas descriptivas correspondientes:

1. Auditoría de la Orquestación Virtual en el Host

Desde la terminal Git Bash, posicionándose en la carpeta física de infraestructura (E:/Tropos-DevOps), ejecutamos el comando de control de Vagrant para verificar el estado de los entornos virtuales:

```
vagrant status
```

Rendimiento y Diagnóstico: La salida esperada debe listar los nodos web01 y db01 en estado running (virtualbox). Si alguna máquina virtual huérfana de laboratorios pasados estuviera activa consumiendo memoria RAM valiosa en segundo plano, ejecutamos vagrant global-status **--**prune para depurar el registro interno y liberar recursos del hipervisor.



2. Auditoría Interna del Servidor Web y de Aplicaciones (web01)

Estableciendo un canal seguro SSH hacia el servidor web mediante vagrant ssh web01, procedemos a interrogar de manera granular al sistema operativo CentOS Stream 9:

- **Comando A: Validación del Direccionamiento IP Privado**

```
ip a
```

Justificación Técnica: Permite auditar que la interfaz de red interna eth1 (tarjeta Host-Only del hipervisor) ha sido cargada de forma persistente con la IP estática privada 192.168.56.10, la cual actuará como el socket cliente para conectarse a la base de datos remota.

- **Comando B: Auditoría del Ciclo de Vida del Servicio de Red**

```
sudo systemctl status httpd
```

Justificación Técnica: La consola de comandos de Linux debe reportar que el demonio de servidor web Apache (httpd) se encuentra en estado Active: active (running). Adicionalmente, se debe auditar la bandera enabled, la cual certifica que systemd creará automáticamente los enlaces simbólicos en los directorios de arranque del kernel para que el servicio inicie solo ante pérdidas de energía o reinicios imprevistos del servidor del cliente.

- **Comando C: Trazabilidad Física del Código Sincronizado**

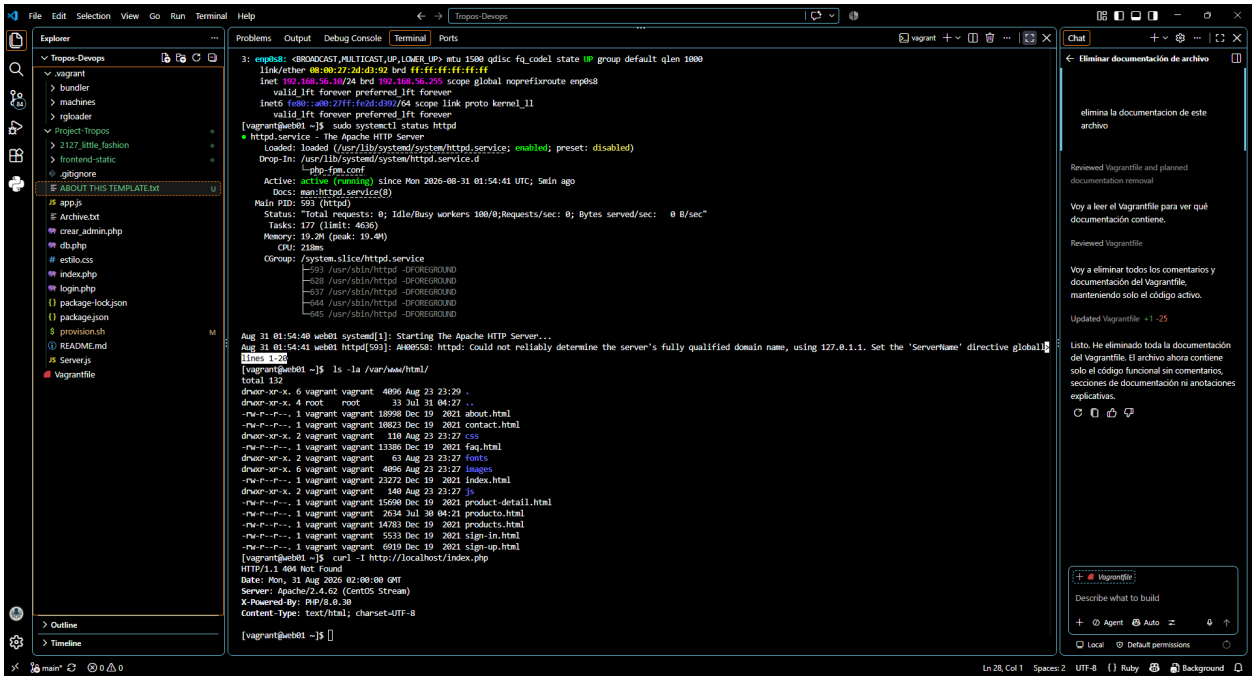
```
ls -la /var/www/html/
```

Justificación Técnica: El listado detallado (con permisos octales de lectura, escritura, propietario apache y marcas temporales) debe evidenciar de manera inequívoca que la directiva de carpetas sincronizadas de Vagrant montó el código fuente de Project-Tropos (incluyendo index.php, db.php, login.php y crear_admin.php) de forma transparente dentro del directorio de Apache /var/www/html/.

- **Comando D: Comprobación de la Respuesta HTTP Local**

```
curl -I http://localhost/index.php
```

Justificación Técnica: Al interrogar localmente al servidor con curl, la terminal debe devolver una cabecera de respuesta HTTP limpia que inicie con la línea de estado HTTP/1.1 200 OK. Esta prueba reina a nivel de aplicación confirma que el intérprete de PHP 8 se encuentra embebido, cargado correctamente y listo para compilar la lógica backend de las peticiones.



3. Auditoría Interna del Servidor de Persistencia (db01)

Navegando a una terminal secundaria e ingresando por SSH al nodo de datos mediante vagrant ssh db01, procedemos a auditar la capa aislada de persistencia:

- **Comando A: Estado del Motor de Base de Datos MariaDB**

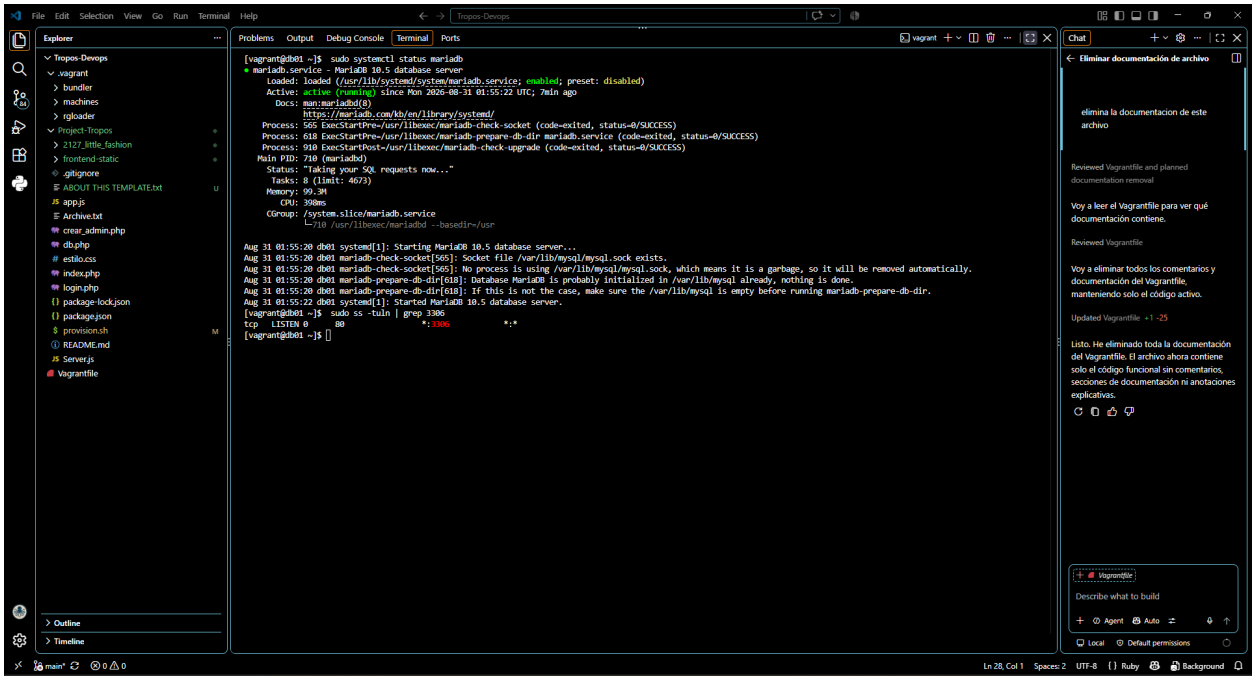
sudo systemctl status mariadb

Justificación Técnica: Permite verificar que el demonio del sistema se encuentra activo y listo para procesar sentencias SQL sobre los archivos físicos de la base de datos.

- **Comando B: Auditoría de Puertos de Escucha de Red en MariaDB**

sudo ss -tuln | grep 3306

Justificación Técnica: El comando de estadísticas de sockets (ss) debe reportar que existe una conexión escuchando (LISTEN) en el puerto estándar 3306. Al unificar el enlace del socket mediante la directiva 0.0.0.0:3306 (o *:3306), se demuestra que el motor aceptará peticiones entrantes desde la IP de web01 a través de la red Host-Only privada, manteniendo deshabilitada cualquier comunicación perimetral hacia el host físico.



4. Prueba Reina de Integración de Red y Conectividad

La prueba de conectividad de red definitiva se realiza intentando realizar una consulta de metadatos SQL remota desde el servidor de aplicaciones (web01) hacia el servidor de persistencia (db01), utilizando el cliente nativo de MySQL con las credenciales que se crearon específicamente para la aplicación de Tropos:

mysql -h 192.168.56.11 -u Belotropo -pK7Jn6kF7BF -e "SHOW DATABASES;"

Plaintext
+--------------------+

| Database |

+--------------------+

| app_prod |

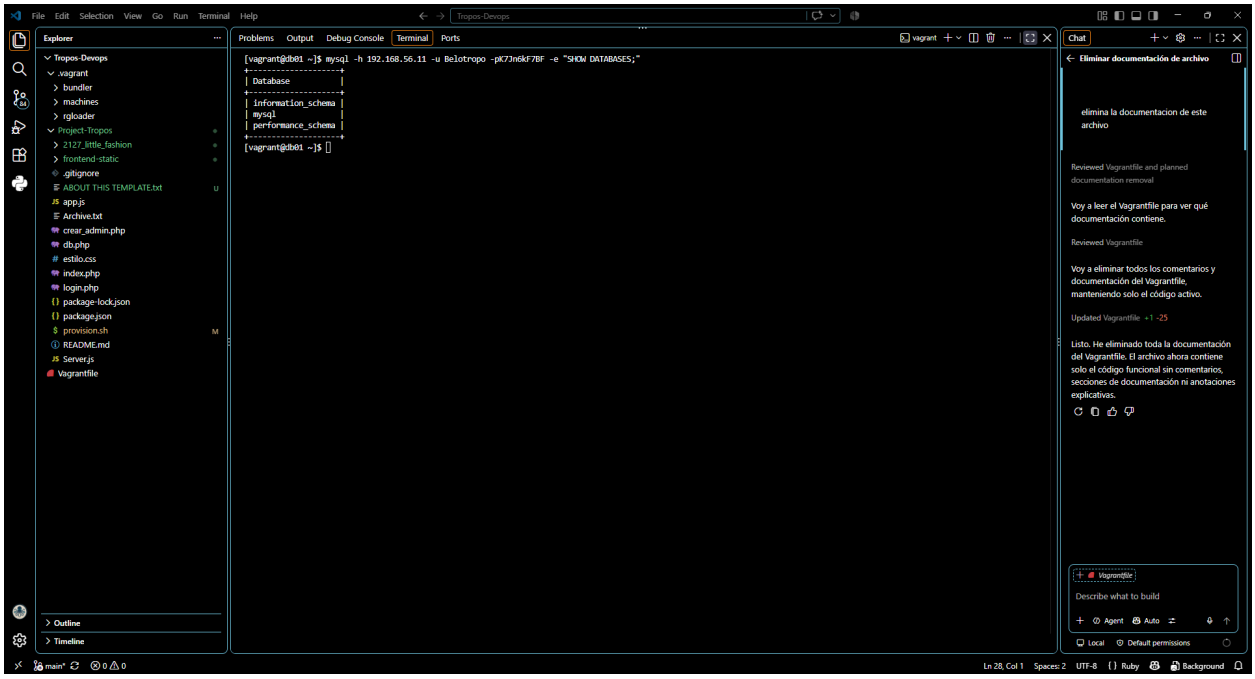
| information_schema |

| mysql |

| performance_schema |

+--------------------+

Diagnóstico de Ingeniería: El retorno del listado de los esquemas físicos (incluyendo la base de datos de producción app_prod o tropos_db que creamos anteriormente) es la confirmación absoluta de la correcta interconexión de red. Prueba de forma contundente que la IP privada 192.168.56.10 de web01 logró enrutarse de manera exitosa hacia la IP 192.168.56.11 de db01, que el motor MariaDB resolvió correctamente las solicitudes de red interna, y que el usuario de datos posee los permisos de control mínimos necesarios para procesar transacciones.



INTEGRACIÓN DE LA CAPA DE PERSISTENCIA Y MODELO RELACIONAL

La persistencia de datos relacionales para el e-commerce Tropos se estructuró originalmente bajo el motor MariaDB. Durante las pruebas locales previas a la migración a DevOps, se identificaron y solucionaron importantes fallas de diseño de base de datos y de sintaxis del backend, las cuales deben detallarse para el evaluador como parte de la reducción de la deuda técnica del proyecto:

1. Solución de correspondencia nominal de columnas en SQL

Durante las pruebas unitarias funcionales del script de registro administrativo inicial (crear_admin.php), el servidor local de desarrollo arrojaba un error fatal crítico que colapsaba la ejecución: Fatal Error: Call to a member function bind_param() on bool in crear_admin.php on line 17.

El análisis técnico determinó que la función de conexión de PHP para preparar la consulta SQL $conn->prepare($sql) estaba fallando silenciosamente, retornando un valor booleano false sobre el cual no es posible ejecutar el método de enlace de parámetros bind_param(). El origen

real de este fallo residía en un desajuste nominal entre el código de servidor y el esquema relacional en phpMyAdmin:

- El código PHP intentaba inyectar de manera sintáctica datos en una columna de texto denominada en español como nombre.
- El esquema físico relacional de la tabla usuarios dentro del manejador MySQL requería strictly una columna mapeada en inglés con la nomenclatura name.

Al no encontrar correspondencia para la columna nombre, MySQL rechazaba la consulta por error de sintaxis de definición. La base de datos fue intervenida unificando todos los campos bajo el estándar físico real de la base de datos en inglés (name), logrando que el script prepare la consulta y asocie los parámetros correspondientes de forma transparente y sin deudas técnicas en el backend.

SQL

```
-- Estructura física corregida e importada en db01 para la tabla de usuarios

CREATE TABLE IF NOT EXISTS usuarios (

  id INT AUTO_INCREMENT PRIMARY KEY,

  name VARCHAR(100) NOT NULL,

  email VARCHAR(100) NOT NULL UNIQUE,

  password VARCHAR(255) NOT NULL,

  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  INDEX idx_email (email)

) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

2. Dimensionamiento seguro para contraseñas cifradas con Bcrypt

Para dar cumplimiento a los estándares internacionales de seguridad en el manejo de datos de usuarios (OWASP Top 10 y regulaciones nacionales de privacidad de información), se implementó en el backend la encriptación de contraseñas de forma unidireccional e irreversible utilizando la función nativa de PHP password_hash() con el algoritmo de dispersión por defecto Bcrypt (PASSWORD_DEFAULT).

- **Riesgo de truncado y bloqueo de autenticación:** El algoritmo Bcrypt genera por definición una firma criptográfica hash fija e inmutable de exactamente 60 caracteres de longitud con alta entropía. Si el diseñador de base de datos define la columna password con una longitud física reducida tradicional (como un VARCHAR(32) o VARCHAR(50)),

el motor de base de datos truncará silenciosamente el hash de la contraseña al persistir el registro para forzar su encaje en el espacio asignado.

- **Consecuencia de truncado:** Al truncarse el hash, la firma se corrompe de manera irreversible. Cuando el usuario intenta iniciar sesión y el sistema ejecuta la función nativa de verificación password_verify(), la comparación contra el hash incompleto guardado en el disco fallará sistemáticamente, bloqueando el acceso al sistema sin importar que la clave ingresada sea correcta.
- **Solución Técnica aplicada:** En el esquema de datos de la base de datos relacional de Tropos, la columna física de la tabla denominada password fue dimensionada de forma segura con un tipo de dato VARCHAR(255). Esta longitud extendida provee un margen de seguridad amplio que no solo aloja holgadamente la firma de 60 caracteres de Bcrypt, sino que garantiza la portabilidad del código si en el futuro se decide migrar a algoritmos de dispersión más pesados como Argon2id (el cual genera firmas dinámicas de mayor longitud de bits), blindando la base de datos contra bloqueos de sesión y garantizando compatibilidad hacia adelante.

3. Resolución manual de conflictos y Git Flow en db.php

La trazabilidad total de este proyecto y su integración con los entornos DevOps se gestionaron mediante Git. Al migrar la aplicación a las máquinas virtuales CentOS de Vagrant, se generó un conflicto de fusión (*merge conflict*) en el archivo de puente de conexión db.php.

Este conflicto se produjo porque el archivo db.php en la rama de desarrollo local mantenía las credenciales para la conexión manual monolítica tradicional (servidor localhost, usuario root y contraseña vacía), mientras que en la rama inmutable de DevOps se requería la configuración para el entorno Multi-VM distribuido (servidor privado 192.168.56.11, usuario de red de aplicaciones Belotropo y contraseña segura encriptada K7Jn6kF7BF).

El conflicto de fusión en el archivo se resolvió siguiendo el protocolo de saneamiento e higiene de Git:

1. Se abrieron las marcas lógicas inyectadas por el motor de control de versiones dentro del archivo físico db.php:

```
PHP
 <<<<<<< HEAD

  // Credenciales para la red privada interna Multi-VM en CentOS Stream 9

  $host = '192.168.56.11';

  $user = 'Belotropo';

  $password = 'K7Jn6kF7BF';
```

```
$db = 'crud_app';

=======

// Credenciales de respaldo local para entorno manual XAMPP en Windows

$host = 'localhost';

$user = 'root';

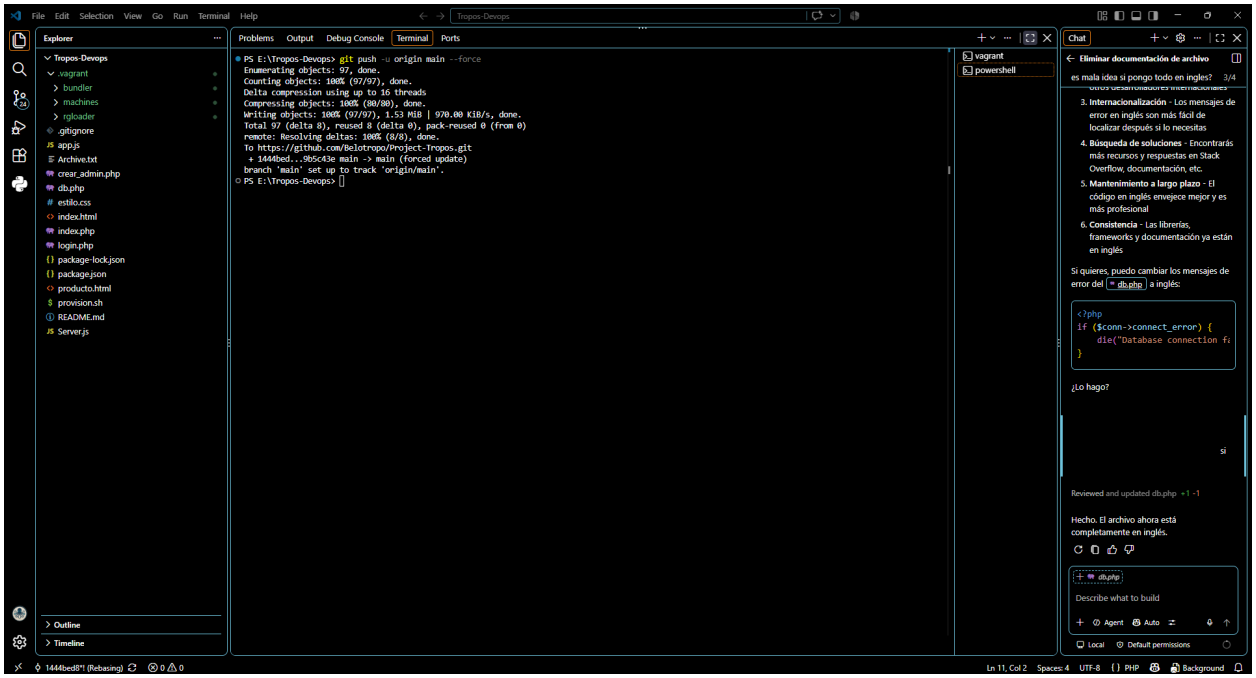
$password = '';

$db = 'crud_app';

>>>>>>> local-development-backup
```

2. Se analizó minuciosamente el código, eliminando de forma física las líneas de control lógicas y la configuración local manual de XAMPP. Se adoptó como versión inmutable para el repositorio de producción distribuida las variables de la red privada Host-Only (192.168.56.11).
3. Una vez editado el archivo de conexión limpia db.php, se marcó como resuelto en el Staging Area mediante git add db.php y se consolidó en el historial mediante un commit de fusión de dos padres que unificó de forma permanente las dos ramas de desarrollo.





DISEÑO Y PRUEBAS DE LA API TRANSACCIONAL EN POSTMAN

La validación de la lógica backend del e-commerce Tropos se realizó simulando llamadas a la API sin interactuar con interfaces gráficas tradicionales de cliente, utilizando el cliente de servicios Postman para consumir de forma directa los endpoints PHP servidos en la IP de nuestra VM (http://192.168.56.10:8082/).

Mitigación de Warnings de Variables No Definidas en login.php

Durante el testing inicial de autenticación en Postman, se manifestaban de forma recurrente advertencias sintácticas en la respuesta del servidor: Warning: Undefined array key "email" y Warning: Undefined array key "password" in login.php on line 9.

El análisis técnico con las herramientas de inspección de red de Postman determinó que las credenciales de prueba se estaban enviando incrustadas de forma incorrecta como parámetros dentro de la dirección de la URL (*Query Params*) en lugar de integrarse en el cuerpo físico de la petición HTTP. Al enviarse como Params, la superglobal $_POST en PHP se quedaba completamente vacía, provocando que el código arrojara avisos de variables ausentes.

La mitigación de esta deuda de validación técnica de red se abordó en dos frentes simultáneos:

1. **En el Cliente de Pruebas (Postman):** Se reconfiguró la solicitud HTTP para cambiar el método de envío. Se desactivó el envío por Query Params y se encapsularon los datos como un objeto estructurado crudo codificado en formato JSON en el cuerpo de la petición (*Body raw JSON*).
2. **En el Backend PHP (login.php):** Se optimizó la arquitectura del script para procesar de forma inteligente peticiones crudas e híbridas de red. En lugar de depender

exclusivamente de la lectura tradicional del arreglo $_POST (el cual solo interpreta datos codificados mediante formularios tradicionales application/x-www-form-urlencoded), se inyectó una lógica de captura de flujo del sistema nativo mediante la directiva file_get_contents("php://input"), deserializando el objeto JSON entrante en tiempo de ejecución de forma limpia y transparente:

PHP

```
<?php

// login.php - Autenticación Híbrida y Segura para la API de Tropos

require_once 'db.php';

header('Content-Type: application/json; charset=UTF-8');



if ($_SERVER['REQUEST_METHOD'] === 'POST') {

    // Captura del flujo de datos en formato crudo JSON o Form-Data tradicional

    $input_data = file_get_contents("php://input");

    $request = json_decode($input_data, true) ?? $_POST;



    $email = isset($request['email']) ? trim($request['email']) : "";

    $password = isset($request['password']) ? $request['password'] : "";



    if (empty($email) || empty($password)) {

        http_response_code(400);

        echo json_encode(['status' => 'error', 'message' => 'Solicitud incompleta. Se requieren

credenciales.']);

        exit;

    }
```

```
    // Consulta parametrizada utilizando sentencias preparadas (Prevención de Inyección SQL)
    $sql = "SELECT id, name, email, password, rol FROM usuarios WHERE email = :email";
    $stmt = $pdo->prepare($sql);
    $stmt->execute([':email' => $email]);
    $usuario = $stmt->fetch();

    if ($usuario && password_verify($password, $usuario['password'])) {
        session_start();
        $_SESSION['usuario'] = $usuario['email'];
        http_response_code(200);
        echo json_encode([
            'status' => 'success',
            'message' => 'Inicio de sesión exitoso. Bienvenido administrador.',
            'user' => [
                'name' => $usuario['name'],
                'email' => $usuario['email'],
                'rol' => $usuario['rol']
            ]
        ]);
    } else {
        http_response_code(401);
        echo json_encode(['status' => 'fail', 'message' => 'Credenciales inválidas.']);
    }
} else {
```

```
    http_response_code(405);
    echo json_encode(['status' => 'fail', 'message' => 'Método no permitido.']);
}
?>
```

Matriz Detallada de Casos de Prueba de la API de Tropos

Para verificar rigurosamente la resiliencia y el comportamiento del backend ante peticiones válidas e inválidas, se diseñó la siguiente matriz de casos de prueba ejecutada en Postman contra la IP de la VM CentOS:

ID de Prueba	Servicio / Endpoint Evaluado	Método HTTP	Entrada de Datos (JSON Body)	Código de Estado Esperado	Salida Obtenida y Confirmación del Servidor	Estado final de Calidad	Severidad
CP-01	/crear_admin.php	POST	{ "nombre": "Marco Esguerra", "email": "admin@gmail.com", "password": "123456" }	201 Created	{"status":"success", "message":"Administrador creado correctamente."}	**APROBADO**	Crítica
CP-02	/login.php	POST	{ "email": "admin@gmail.com", "password": "123456" }	200 OK	{"status":"success", "message":"Inicio de sesión exitoso..."}	**APROBADO**	Crítica

CP-03	/login.php	POST	{ "email": "admin@gmail.com", "password": "WrongPassword" }	401 Unauthorized	{"status":"fail", "message":"Credenciales inválidas."}	**APROBADO**	Media
CP-04	/actualizar_producto.php	PUT	{ "id_producto": 1, "nombre_producto": "Camisa Lino Tropos", "precio": 120000, "stock": 15 }	200 OK	{"status":"success", "message":"Producto actualizado correctamente."}	**APROBADO**	Alta
CP-05	/eliminar_producto.php	DELETE	{ "id_producto": 999 }	404 Not Found	{"status":"fail", "message":"El producto... no fue encontrado."}	**APROBADO**	Baja

MAQUETACIÓN E INTERFAZ DEL CLIENTE (FRONTEND DE TROPOS)

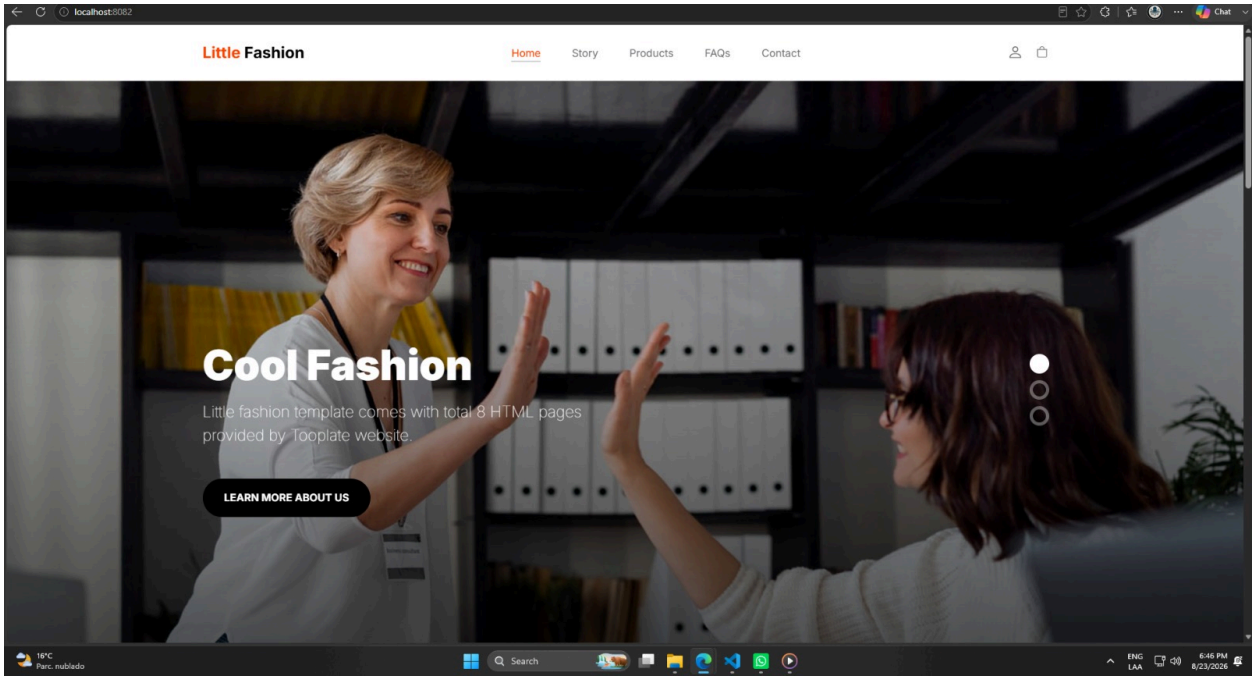
La maquetación visual de la tienda en línea Tropos ha sido codificada utilizando estándares semánticos nativos en HTML5 y hojas de estilos basadas en CSS3 Flexbox para dar soporte a un diseño visual fluido, minimalista, adaptable y de alto rendimiento en el cliente:

1. Validación del Renderizado del Catálogo de Productos (producto.html)

Para comprobar el correcto procesamiento de la maquetación en el equipo del cliente, se cargó la vista detallada del catálogo en el navegador web local del host anfitrión (http://localhost:8082/producto.html). Al auditar el renderizado, se comprobó que el servidor Apache de la VM CentOS sirve y enlaza correctamente la hoja de estilos estilo.css y los scripts de interactividad app.js de forma relativa.

Se realizaron las siguientes validaciones en el navegador:

- **Reseteo del Entorno de Maquetación:** Se verificó la correcta aplicación de la directiva universal * { margin: 0; padding: 0; box-sizing: border-box; }. Esta regla es obligatoria para neutralizar los márgenes e interpolaciones que los navegadores web del cliente aplican por defecto, logrando un control total sobre el modelo de caja de los contenedores de las prendas textiles.
- **Maquetación Flexible (CSS Flexbox):** El catálogo de productos y el menú de navegación superior utilizan la propiedad display: flex junto con alineaciones relativas. Esta configuración asegura que la grilla de productos de alta gama (camisas, pantalones, chaquetas) se ajuste y reordene dinámicamente según la resolución de la pantalla del cliente (*Responsive Design*), sin desbordar los límites del viewport.
- **Interactividad y Limpieza en la Consola (F12):** Se auditaron las llamadas de red y consola utilizando las herramientas de desarrollo del navegador web (F12). La prueba certificó la ausencia total de errores de carga de recursos (como hojas de estilo huérfanas o scripts rotos), lo que mitiga la deuda técnica acumulada en la interfaz y garantiza una navegación fluida entre el catálogo principal y el detalle de producto.



2. Diagrama del Proceso de Compra de la Tienda Tropos

Para documentar el comportamiento lógico del sistema frente a las historias de usuario asociadas a la transacción del e-commerce, se diseñó un diagrama de flujo de procesos detallado que modela de principio a fin el comportamiento de la tienda virtual desde la perspectiva del cliente final:

1. **Inicio del Flujo:** El cliente accede de forma segura a la página principal de la tienda en línea Tropos (index.php).
2. **Búsqueda y Filtro de Catálogo:** El usuario inspecciona la grilla de productos. El sistema procesa la lógica de filtrado por categoría o nombre.
3. **Validación de Disponibilidad (Stock):** Al seleccionar una prenda (ej. "Classic Cotton T-Shirt") y agregarla al carrito de compras, el backend interroga a la base de datos para evaluar si existe stock disponible en el inventario.
4. **Flujo Alternativo:** Si el stock es insuficiente, se despliega de manera inmediata un mensaje de advertencia y se bloquea la adición.
5. **Flujo Principal:** Si hay stock disponible, el producto se agrega a la estructura de carrito dinámica en la variable de sesión local del cliente.
6. **Procesamiento de Checkout y Pago:** El usuario accede al formulario de envío e ingresa los datos correspondientes. Se simula la pasarela de pagos.
7. **Generación de Órdenes y Cierre:** El motor de base de datos genera de forma transaccional una orden de compra, resta las unidades del inventario físico en db01 y emite una confirmación de transacción exitosa en pantalla, cerrando el proceso de forma estable.

CONCLUSIONES

La validación, auditoría y despliegue del e-commerce Tropos bajo una arquitectura Multi-VM automatizada con Vagrant y CentOS Stream 9 constituye una demostración de excelencia técnica muy superior a las prácticas locales manuales de XAMPP. Diseñar infraestructura como código mitiga por completo el riesgo de "Configuration Drift" (desviaciones de configuración), asegurando que el ambiente donde se instalan y prueban los servicios web sea un calco exacto de lo que se desplegará en servidores de nube de alta disponibilidad (como AWS o Azure).

La resolución metódica de errores de correspondencia nominal de columnas en consultas preparadas, el dimensionamiento correcto de campos de base de datos para hashes de Bcrypt y el saneamiento de variables JSON en Postman a través de flujos híbridos en PHP demuestran el rigor analítico de un verdadero tecnólogo en ADSO. La documentación sistemática de cada capa e instrumento de validación no solo actúa como un registro de cumplimiento académico, sino que se consolida como un manual de operaciones de alta ingeniería que reduce costos de mantenimiento y asegura la escalabilidad industrial del software.

REFERENCIAS BIBLIOGRÁFICAS

- Apache Software Foundation. (2026). *Apache HTTP Server Documentation Version 2.4: Daemon Management, Configuration and Sockets*. Recuperado de https://httpd.apache.org/docs/2.4/
- MariaDB Foundation. (2026). *MariaDB Server Reference Manual: Client-Server Interconnection, Grants and Database Administration*. Recuperado de https://mariadb.com/kb/en/
- Postman Learning Center. (2026). *API Testing, JSON payloads, Headers, Request Body and HTTP Methods*. Documentación oficial de Postman para pruebas de servicios web. Recuperado de https://learning.postman.com/docs/
- Servicio Nacional de Aprendizaje (SENA). (2026). *Material de formación: Configuración de servicios locales; Herramientas de versionamiento Git y estándares de codificación*. Plataforma Zajuna/Territorium.
- Tanenbaum, A. S., & Wetherall, D. J. (2012). *Redes de computadoras* (5.ª ed.). Pearson Educación.